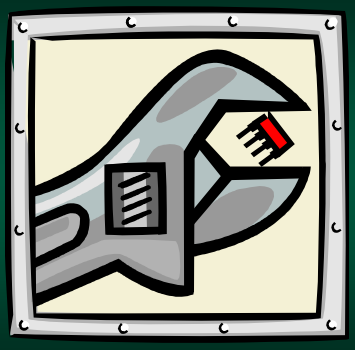
Linked List Data Structure

Part 2

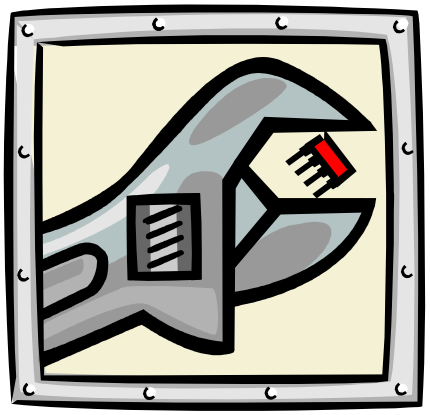


Data Structures

Return to CSC 15 and CSC 20

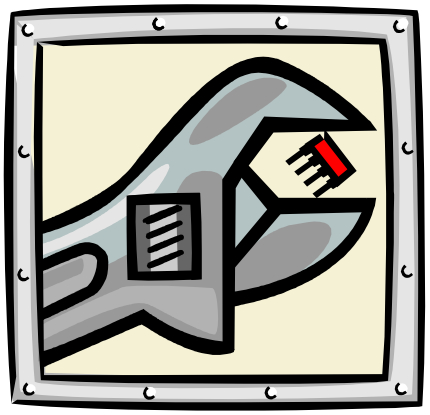
Data Structures

- Arrays and linked-lists are both examples of *data structures*
- These are different techniques of storing and organizing data
- In other words, this is *how* data is stored



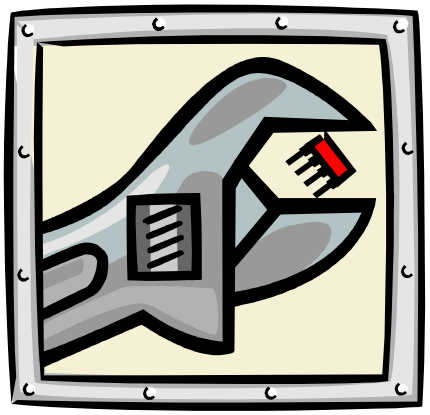
Data Structures

- Depending on *how* data is accessed, some data structures can either excel and falter
- This is true of both arrays and linked lists



Data Structures

- We will do a quick review of arrays and linked lists
- There are more data structures than these two
- We will cover them this semester – some which have incredible in features



Array Data Structure

Hidden math = easy code

Array Data Structure

- The array data structure is found in practically every programming language
- This is also one of the fundamental ways data is stored in memory

Behind the Scenes…

- Arrays are just continuous blocks of memory containing multiple instances of the same type
- Since the instances are continuous, values can be accessed randomly in **O(1)**

Array Math Example: 64-bit int

- Let's assume the array starts at address 2000
- Each array element will take 8 bytes (for 64-bit integers)
- Array elements are stored continuous

2000	446576696E20436F
2008	6F6B000000000000
2016	53616372616D656E
2024	746F205374617465
2032	4353419395000000

Array Math Example: 64-bit int

- `array[0]` is 2000
- `array[1]` is 2008
- `array[2]` is 2016
- `array[3]` is 2024
- `array[4]` is 2032
- etc…

2000	446576696E20436F
2008	6F6B000000000000
2016	53616372616D656E
2024	746F205374617465
2032	4353419395000000

Behind the Scenes…

- So, when an array element is read, internally, a mathematical equation is used
- It uses the start array, the array index, and the size of each element

```
Start Address + (Index × Element Size)
```


Behind the Scenes…

- *This is why the C Programming Language uses zero as the first array element*
- If zero is used with this formula, it gets the start of the array

```
Start Address + (Index × Element Size)
```

Array Element Access

- Notice that the formula **doesn't** use the size of the array
- In fact, this math works the same for any size
- So, reading and writing elements of an array is **O(1)**

Auxiliary Storage in arrays

- Also, because elements are calculated, there is **no** extra storage overhead based on the array size
- So, the *auxiliary storage* overhead is **O(1)**

Auxiliary Storage in arrays

- Note: This mathematical method **only** works if the items are consecutive
- If we want to add an item, we have to make sure it is located physically at the end of the array

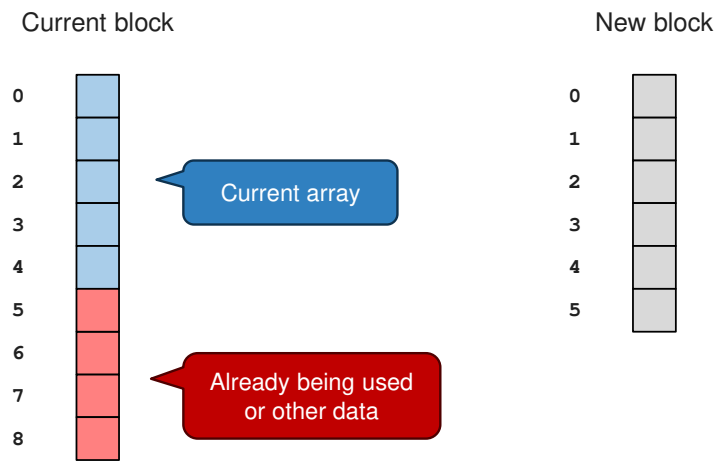
Dynamically Allocated Arrays

- A *dynamically allocated array (aka dynamic array)* is resized anytime an object is added or removed
- Because arrays require all elements to be stored consecutively…

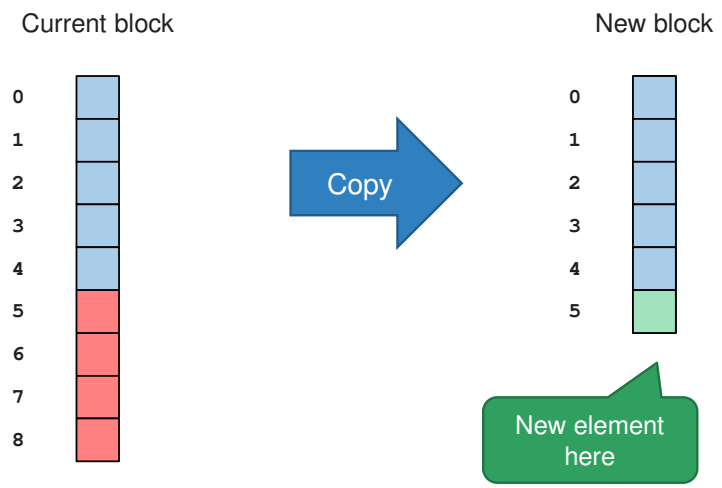
Resizing Arrays

- …the old block of memory (old array) needs to be copied to a new one
- This is extremely costly in both time and resources

Arrays in Memory



Copy Values to New Block



Resizing Arrays is O(n)

- While reading / writing elements takes only **O(1)** …
- … every time an array is resized, it will require **O(n)** time to copy the old array to the new one

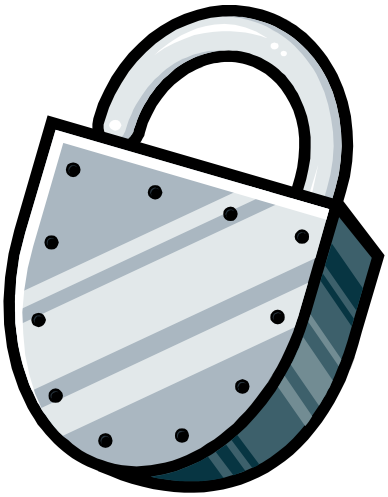
Fixed Capacity Arrays

The size of locked

21

Fixed Capacity Arrays

- The **O(n)** time requirement to copy an array is bad
- …and it cannot be avoided given how arrays work
- However, there are a few alternatives that try to avoid resizing



22

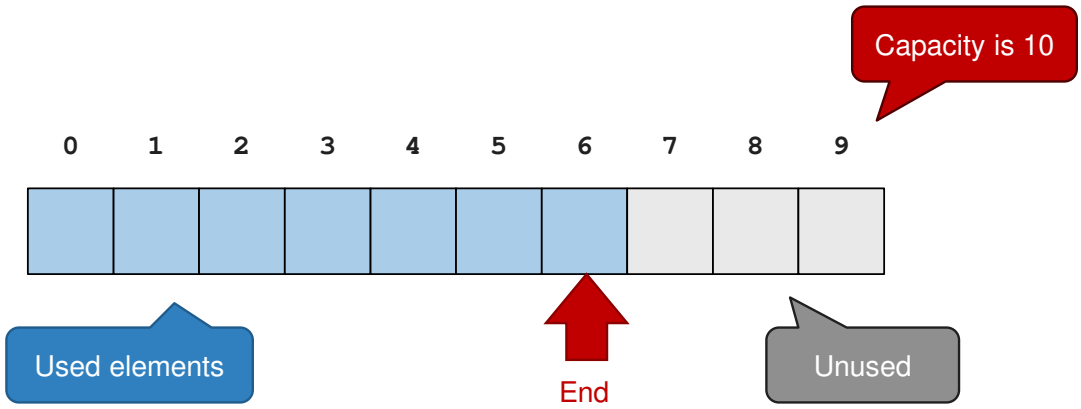
Fixed Capacity Arrays

- Arrays can also have a fixed number of elements called the *capacity*
- So, the array is **never** resized and often only partially filled
- This type of array overcomes the O(n) weakness of dynamic arrays

Fixed Capacity Arrays

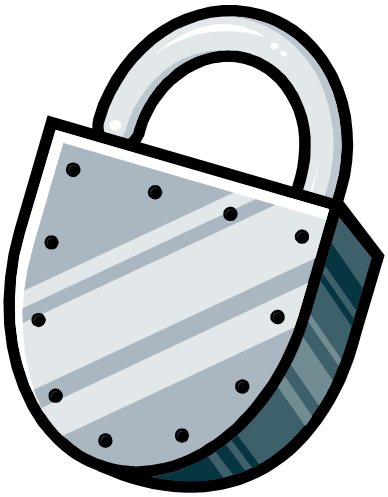
- This type array is called a *Fixed Capacity Array*
- Also known as:
  - *fixed-size array*
  - *partially filled array*
- A variable is used to keep track of the "end" index - i.e. the last array element containing data
- The remainder of the array elements are unused.

Fixed Capacity Array Example



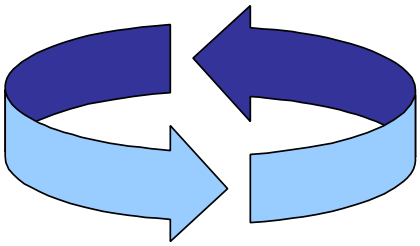
Terminology

- So, the array's *capacity* is the actual count of array elements
- The array's *size* is the number of valid array elements – i.e. the value of the end pointer.



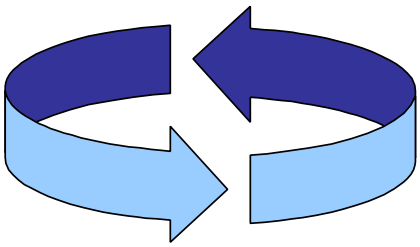
Wrapping Around

- Sometimes, you might need an array that *wraps*
- These are useful if both the first and last items can be removed
- … or older items can be discarded if space is needed

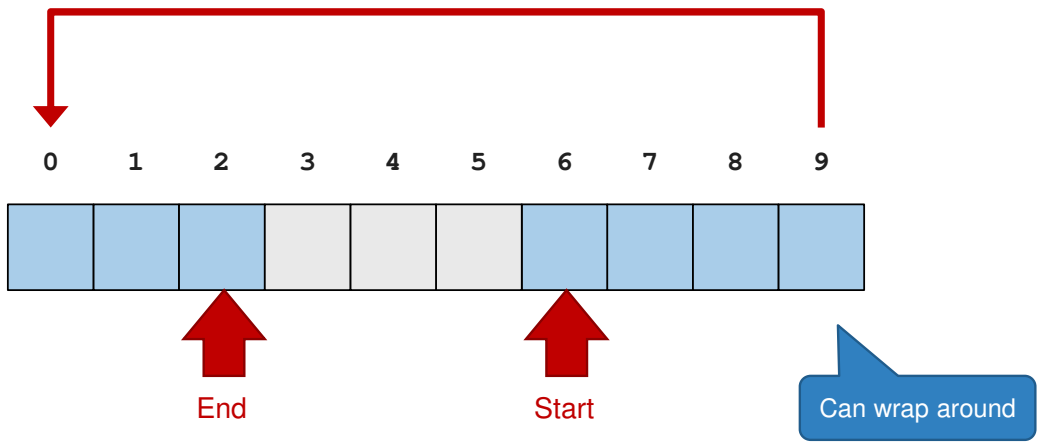


Wrapping Around

- In addition to a "end" index, a "start" index is maintained
- Once the end of the array is reached, the array "wraps" to index 0
- … and continues until end is reached



Wrapping Around



Wrap-Around Equation

- Even though the wrap-around solution looks like it needs a loop – it doesn't
- Actual array index can be computed from the logical index (where 0 is the first item) below

```
actual = (logical + start) % capacity
```

Fixed Capacity Array Benefits

- Randomly reading/writing an fixed capacity array is still **O(1)**
- And, since the array is never resized, adding/removing is also **O(1)**

Fixed Weeknesses Array Benefits

- However, there is a explicit limit on how much data can be stored
- And, some memory is usually wasted (the empty elements)

ArrayList Approach

A Compromised Approach

ArrayList Approach

- Resizing an array every time an element is added/removed is slow: **O(n)**
- The alterative, fixed capacity, limits the size of the array, but maintains **O(1)**

ArrayList Approach

- Another approach is to use a partially filled array – *that can grow*
- When the array runs out of room, then we resize it to a <u>much larger</u> capacity

ArrayList Approach

- So, the rray grows and shrinks in blocks of array elements
- There is some advantages and some disadvantages to this type of approach

ArrayList Data Type

- Most modern programming language libraries include an ArrayList class
- Each language implements them a bit differently
- And this may change when a new version is released.

Java Example

- The Java Programming Language increases the capacity by 1.5× the old capacity
- Decreasing it must be done manually

```
New Capacity = Old Capacity * 1.5
```

Java Example

- This means, when the capacity is reached, it grows by continuously larger increments
- Java ArrayLists start with a capacity of 10.

```
10 → 15 → 22 → 33 → 49 → 73 → ...
```


Java Example

- As you can imagine, the array has to be resized less frequently as it grows
- But, the amount of wasted space also grows

```
10 → 15 → 22 → 33 → 49 → 73 → ...
```

ArrayList Benefits

- Randomly reading/writing an ArrayList is still **O(1)**
- Also, because it a partially filled array, adding items is often **O(1)**

ArrayList Weaknesses

- If an item added/removed, it may trigger the capacity to be updated
- A new array is created (with the new capacity) and the old array is copied to the new one
- So, sometimes, adds are **O(n)**

Linked Lists

Chain of Data

43

Linked Lists

- The array (and the ArrayList) are just two, of many, ways of storing a collection
- *Linked lists* uses a series of "linked" instances to store a collection

44

How Does It Work?

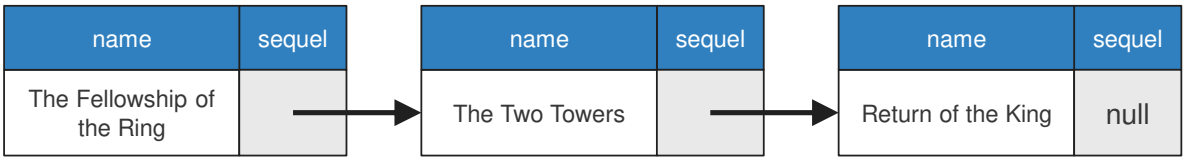
- Since a variable can contain either an instance reference or null, we can something quite clever
- An instance can contain a reference to another instance – *of the same class*
- This creates a chain of connected instances.
- It ends when the "link" is null

How Does That Work?

```
class Book
{
    public String name
    public Book sequel
}
```

A reference to another Book instance

Chain of Books



Chain of Instances

- Notice that the last example is essentially storing Strings
- Can we store other things?
- Yes! This is very simple approach to store any type of data

Chain of Instances

- Each link in our chain, that stores a piece of information, is called a *Node*
- The definition of a Node is extremely simple: data and a link to the next node

Generic Node Class

```
public class Node
{
    public Object data
    public Node next
}
```

We can make this another type

Creating a List (not well, though)

```
Node list = new Node()

list.data = "Rat"
list.next = new Node()

list.next.data = "Cat"
list.next.next = new Node()

list.next.next.data = "Dog"
list.next.next.next = null
```


Constructors Will Help

```
public Node(Object initData, Node initNext)
{
    this.data = initData
    this.next = initNext
}

public Node(Object initData)
{
    this.data = initData
    this.next = null
}
```

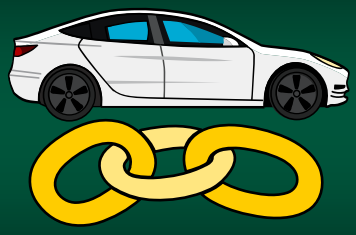
Constructors Do Help

- Constructors do help
- Though it is still a tad hard to read

```
Node list = new Node("Rat", new Node("Cat", new Node("Dog")))
```


We Could Also Do This…

```
Node rat = new Node("Rat")
Node cat = new Node("Cat")
Node dog = new Node("Dog")

rat.next = cat
cat.next = dog
```



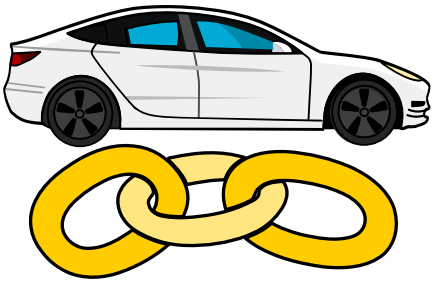
Traversing a Linked List

Chain of Data

55

Traversing a Linked List

- Unlike arrays, where the element can be found using a calculation, linked-lists require the list to be traversed
- This is typically done using a while loop and variable representing the current node



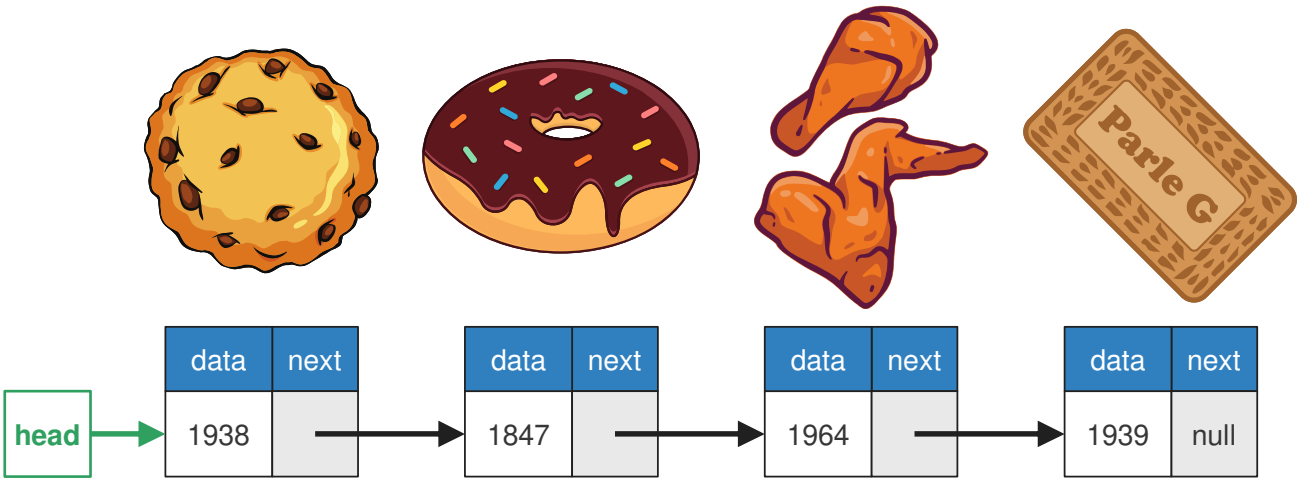
56

Node for Integers

```
public class Node
    public int data
    public Node next
end class
```

Let's use int's for now

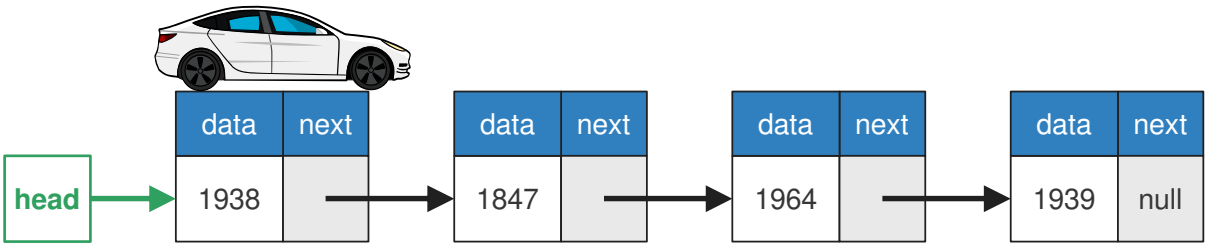
Let's Try This List



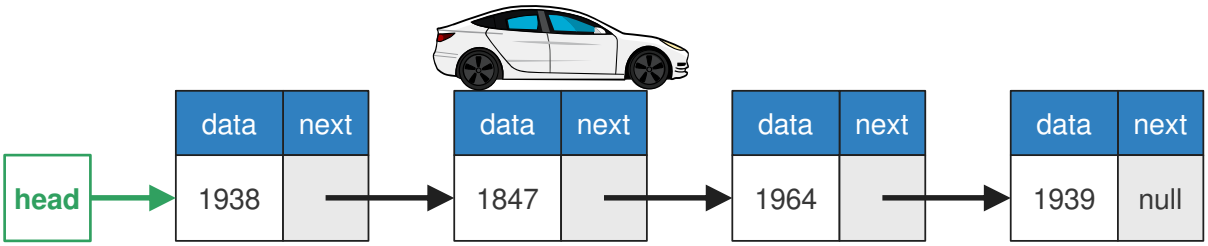
While Loop – Follow the Links

```
current = head

while (current != null)
    print current.data
    current = current.next   ...Go to next node
end while
```

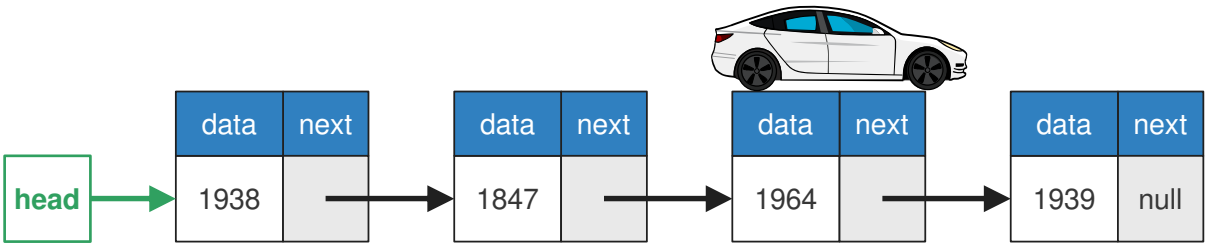

Traversing the List



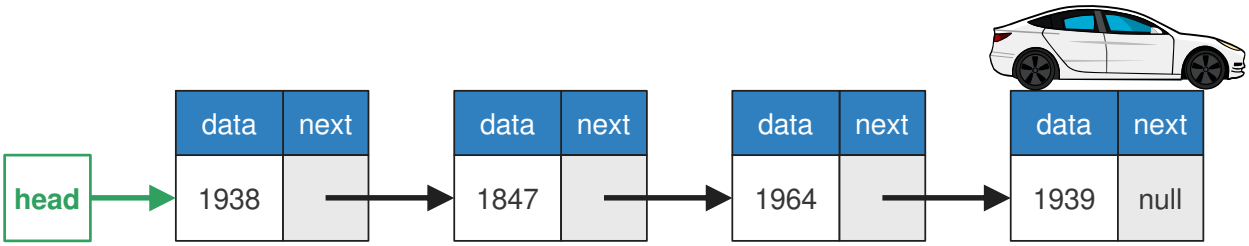
Traversing the List



Traversing the List



Traversing the List



Adding to Linked Lists

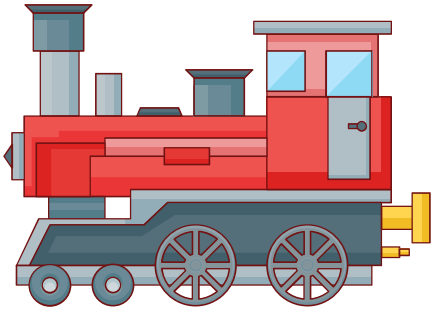
Chain of Data

Adding to Linked Lists

- Adding to Linked Lists is easy to do, but must done with considerable care
- The links (references) need to be updated in a specific order
- … or a link will be lost

Adding to Linked Lists

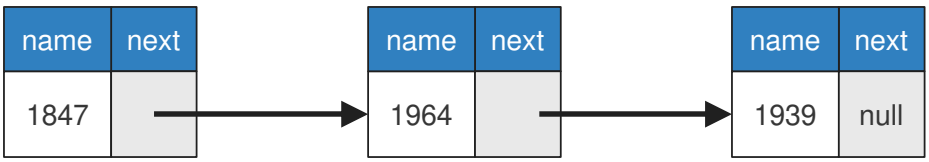
- The first item in a linked list is referred to as the Head (alternatively *Front*)
- The last item, in which the next field is null, is called the *Tail*



Adding to Linked Lists

- In this section, we will add a new node to the front, middle, and end of a linked list
- Most of these actions require just two steps

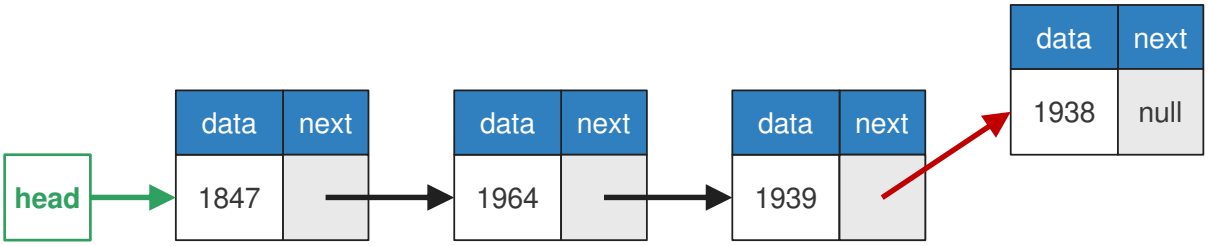
Let's Assume We Have This List



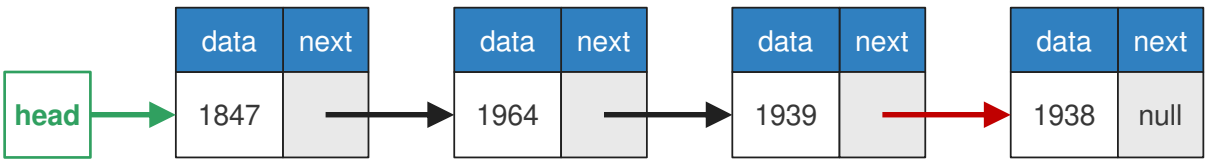
Adding to the Tail of a List

1. Link the tail node (who's next field is null) to the newly added node
2. If a reference to the tail is being maintained, it is linked to the newly added node

Add Tail: 1. Link Tail to the New Node



Add Tail: Resulting List



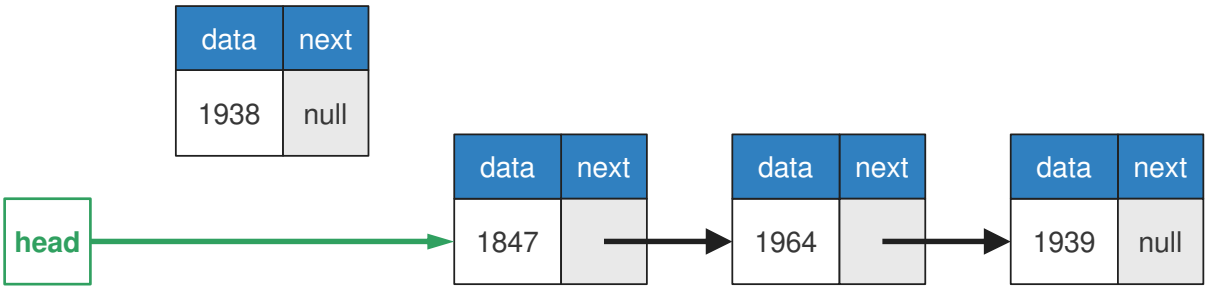
Adding to the Tail of a List

```
... add is the new node
... tail is the last node in the list

tail.next = added
```

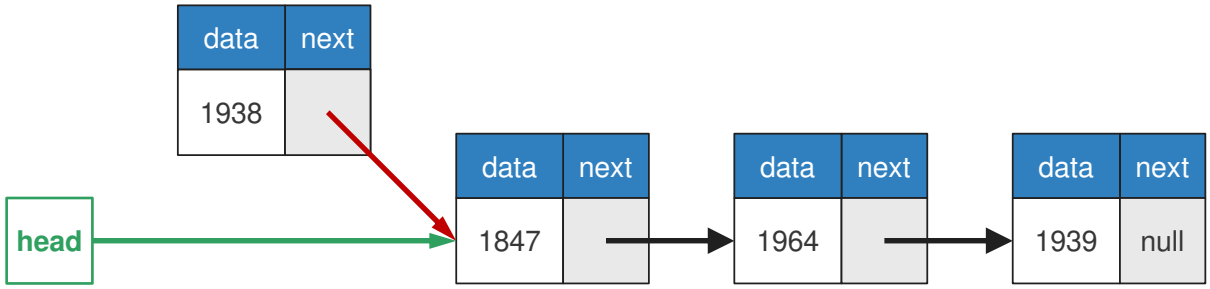
Adding to the Head of a List

1. The newly added node is linked to the head of the list
2. The head is then linked to the newly added node

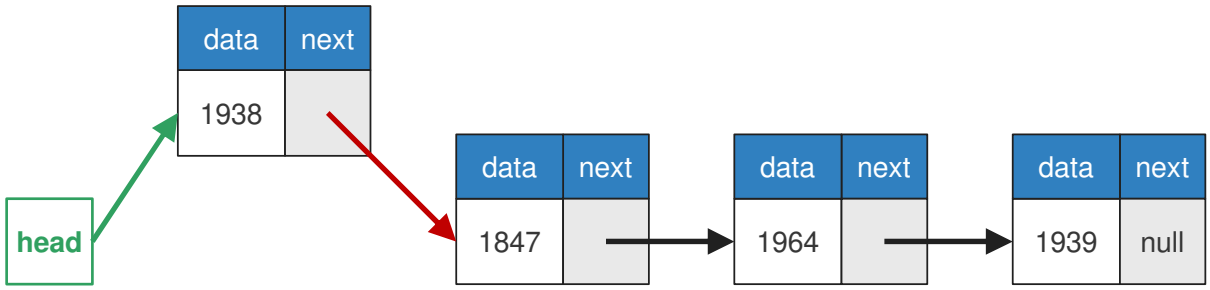
Adding to the Head of a List



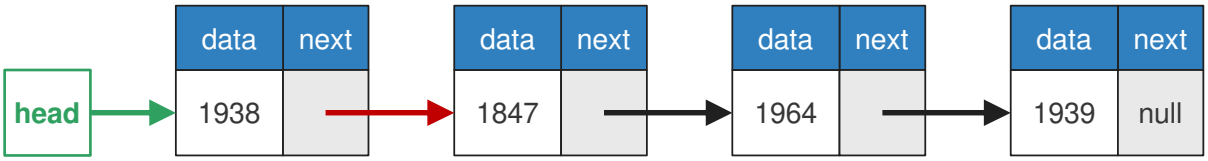
Add Head: 1. Link Node to Head's Reference



Add Head: 2. Set Head Reference to the Node



Add Head: Resulting List



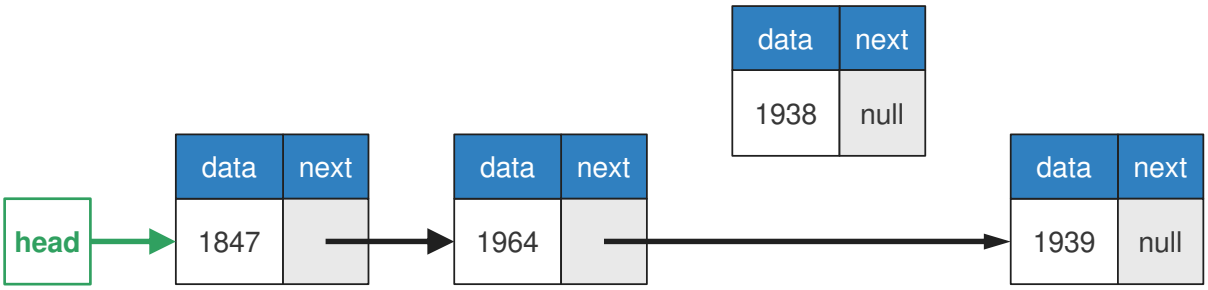
Adding to the Head of a List

```
... 1. Link the new node to the head
added.next = head

... 2. Now, the added item is the head
head = added
```

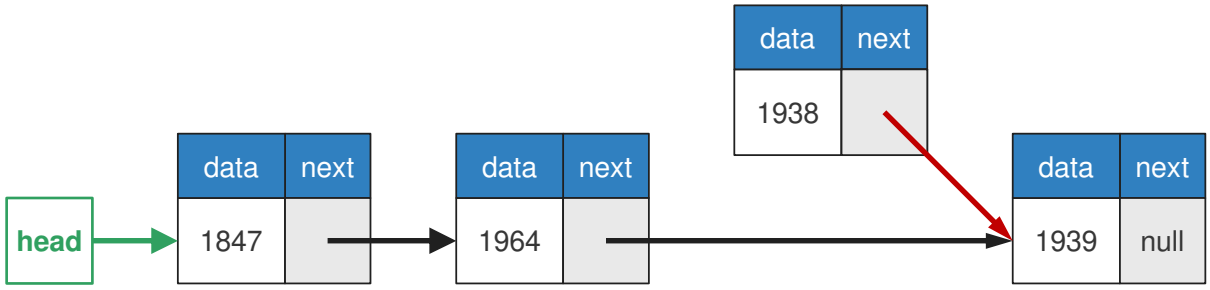
Adding to the Middle of a List

1. The new node is linked to target of the previous node (before where we want to insert)
2. The previous node is then linked to the new node

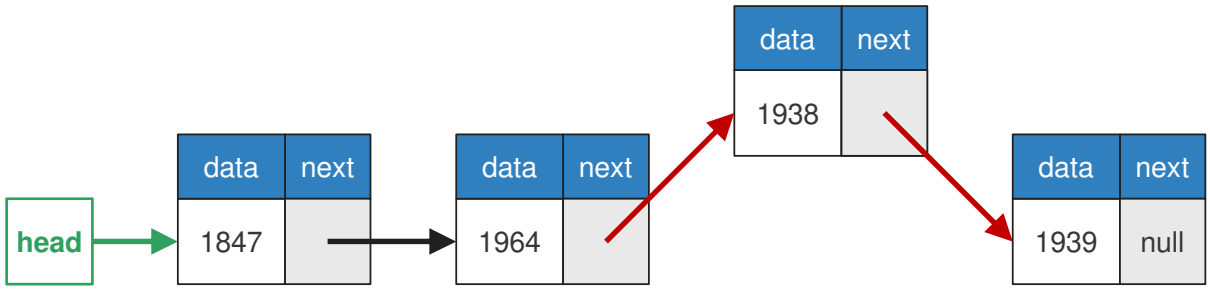
Adding to the Middle of a List



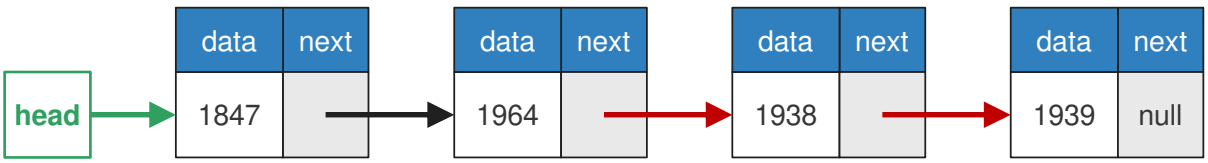
Add Middle: 1. Link Node to Next



Add Middle: 2. Link Previous to the Node



Add Middle: Resulting List



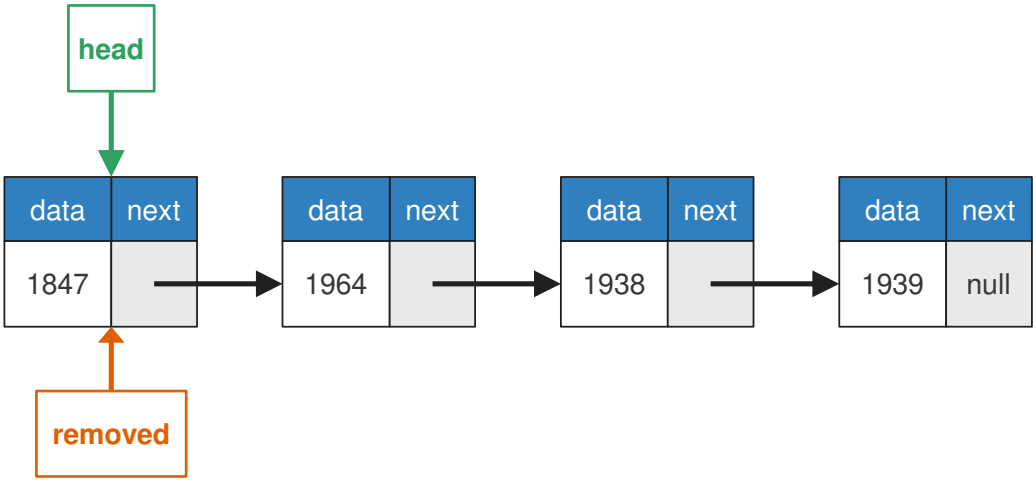
Adding to the Middle of a List

```
... 1. Link the added node to the
       previous node's next
added.next = prev.next

... 2. Link previous to the new node
prev.next = added
```

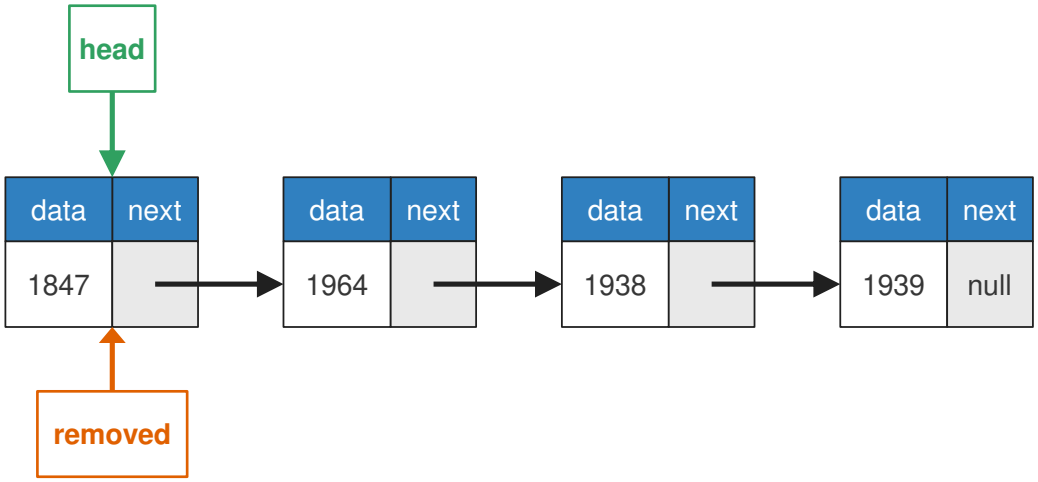
Removing The Head Node

1. Save Link to the Old Head
2. Update the Head Reference to the Head's next link
3. Remove the link from the old head to the new head

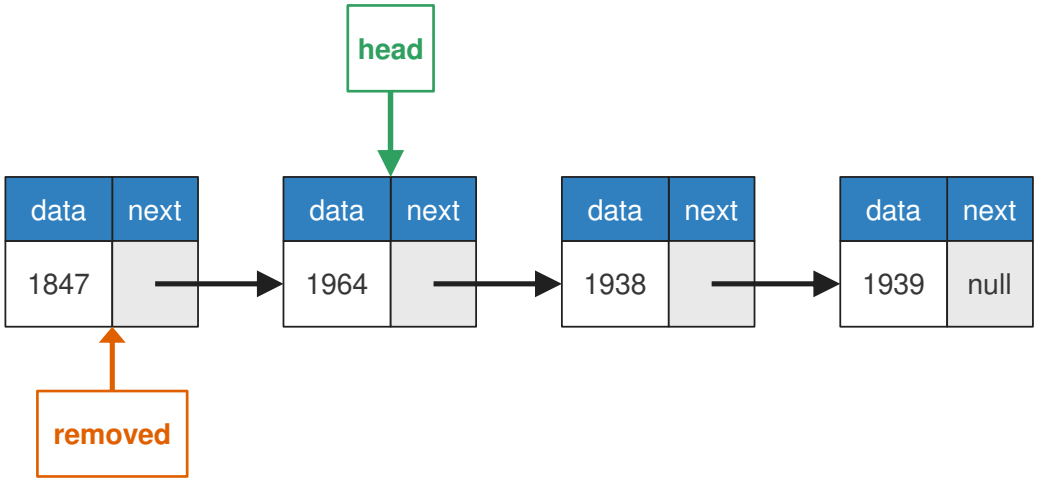
Remove Head: 1. Save Link to the Old Head



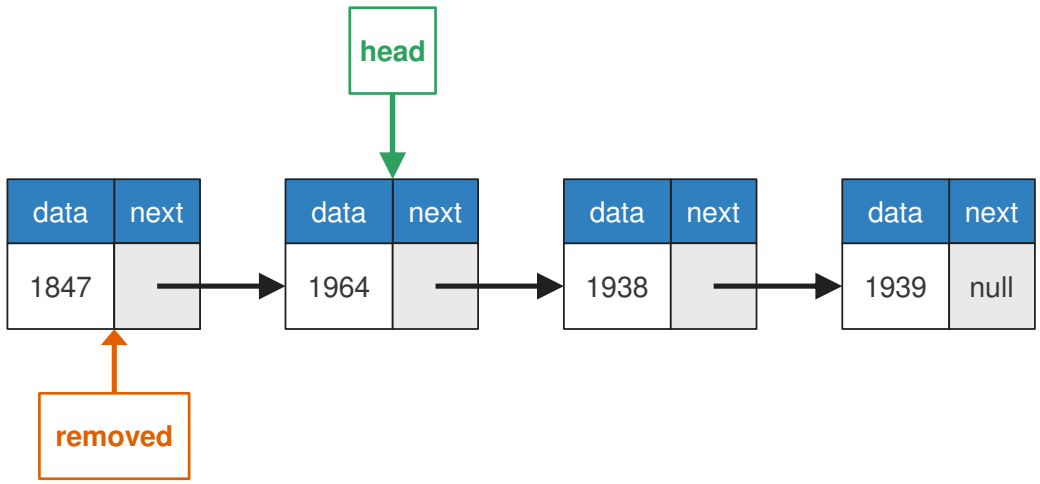
Remove Head: 2. Update the Head Reference



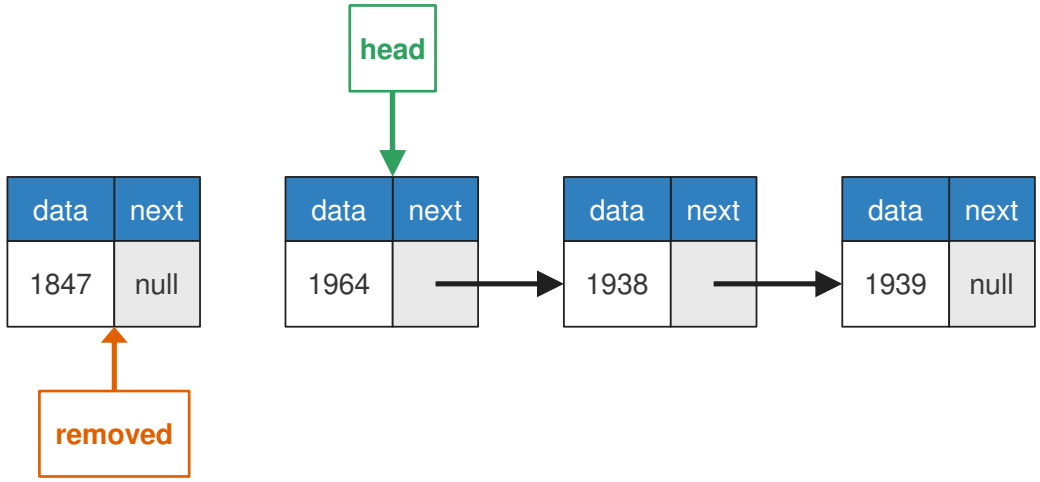
Remove Head: 2. Update the Head Reference



Remove Head: 3. Remove the Link



Remove Head: Complete



Remove Head

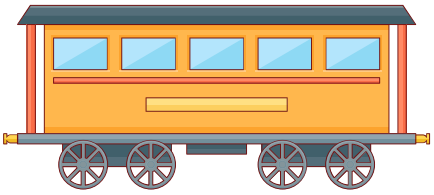
```
... 1. Save a reference to the head
removed = head

... 2. Set head to the head's next link
head = head.next

... 3. Remove link between old head and new head
removed.next = null
```

Maintaining a Tail Node

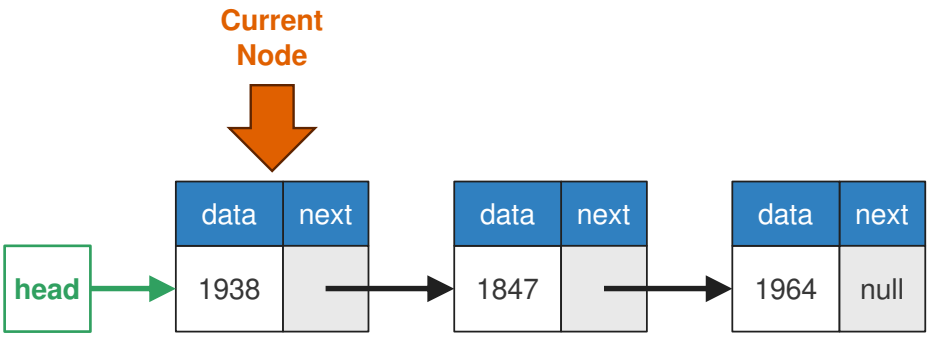
One should keep track the caboose

Head and Tail Nodes

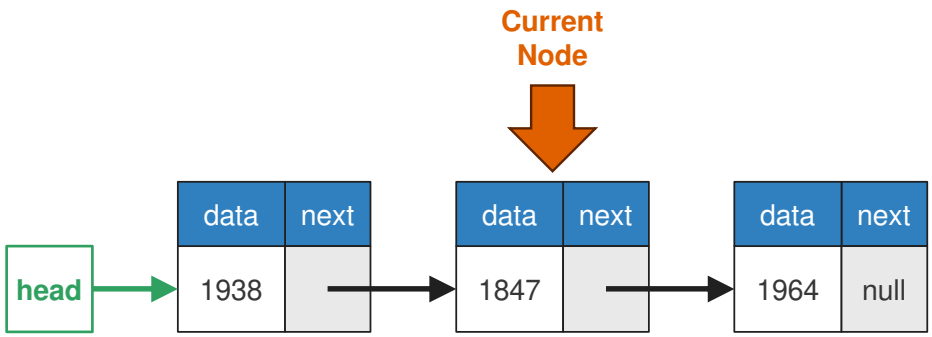
- Linked lists maintain a link to the head node
- Often, in well-written linked lists, a link to the tail node is also maintained
- It is far more efficient



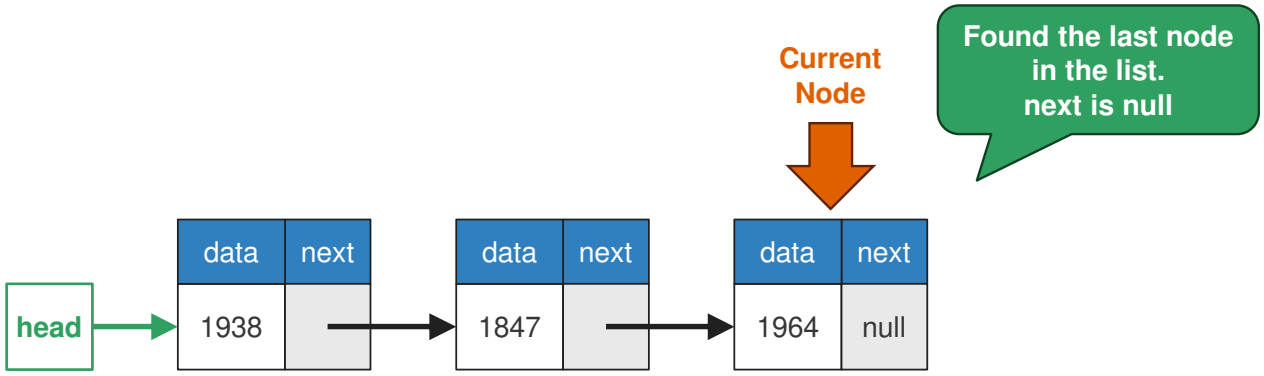
Adding to the End



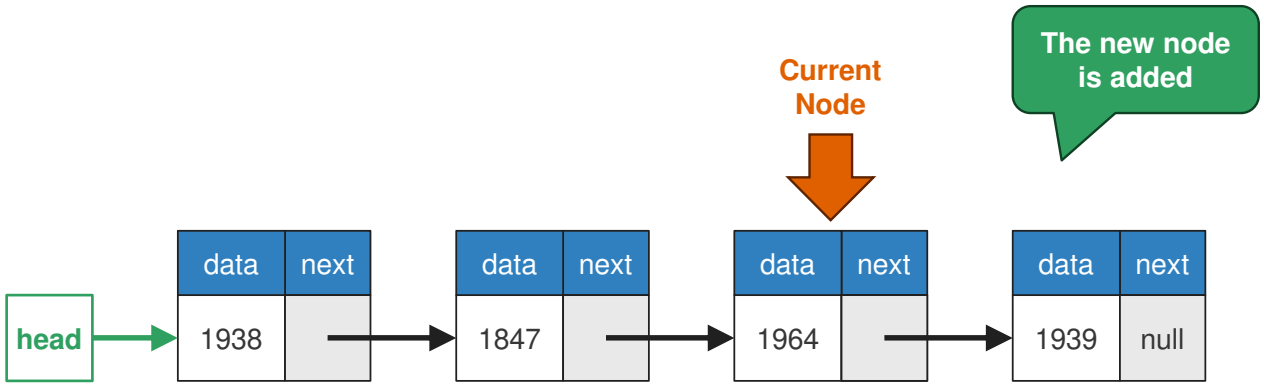
Adding to the End



Adding to the End



Adding to the End

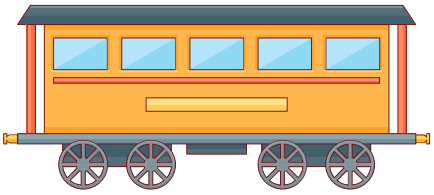


While Loop – Traverse to the End

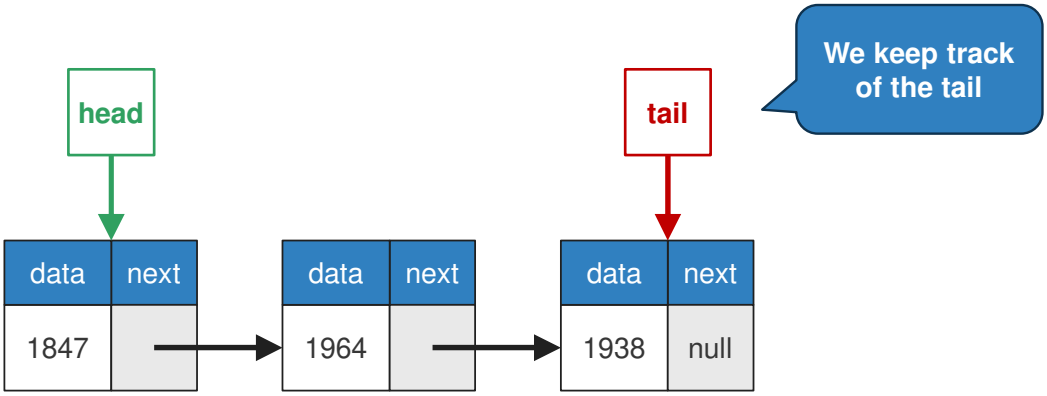
```
current = head
while (current.next != null)
    current = current.next     ... Go to next node
end while

... Current is now the tail. Link to the new node
current.next = added
```

So, that took awhile…

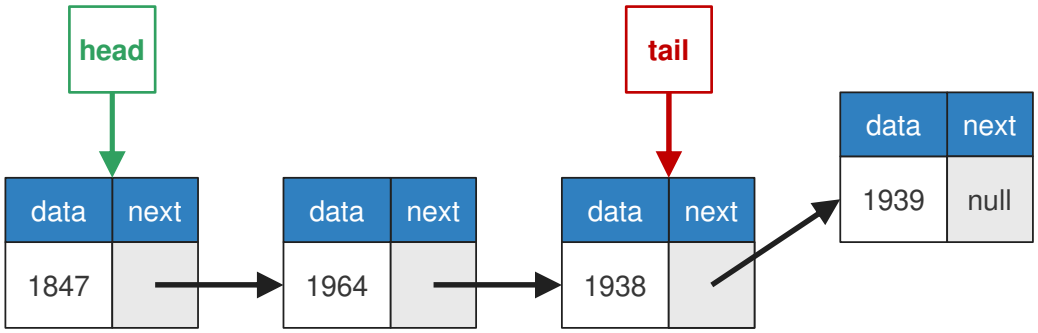
- Notice that, to get the tail now, we had to write loop to traverse all the nodes
- If we *knew* where the tail was beforehand, we wouldn't need a loop



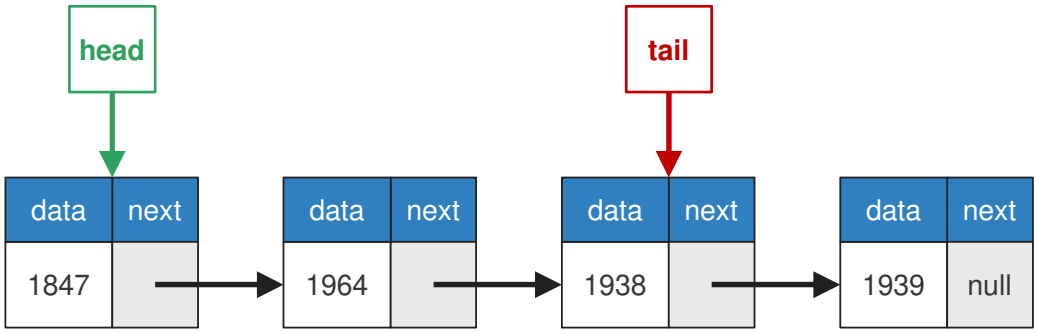
Adding to the End (with a tail)



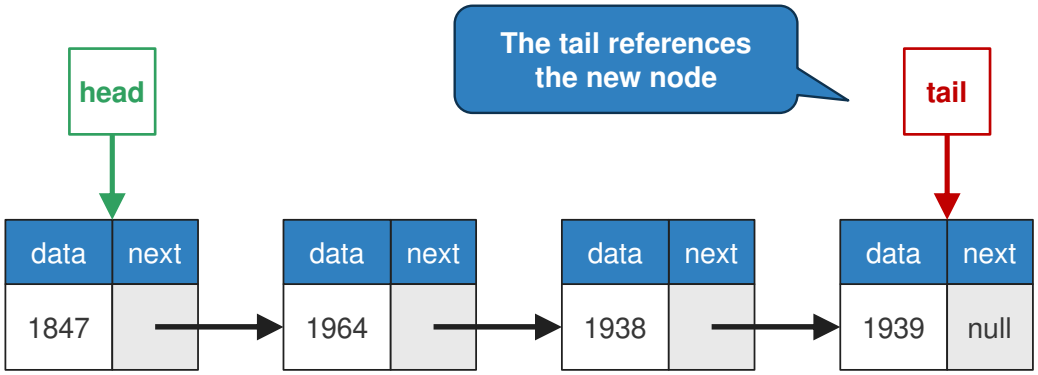
Add Tail: 1. Link Tail to the New Node



Add Tail: 2. Update the Tail Reference



Add Tail: 2. Update the Tail Reference



Adding to the end – with a Tail Node

```
... 1. Link the current tail to the added node
tail.next = added

... 2. The added node is now the tail
tail = added
```

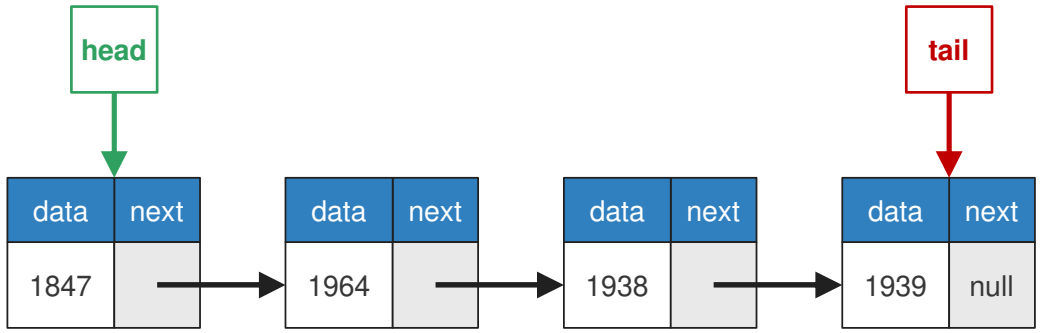
Finding Second-to-Last

- Finding the last item (to add at the end) required a loop
- … or was immediate if we maintained a tail node reference

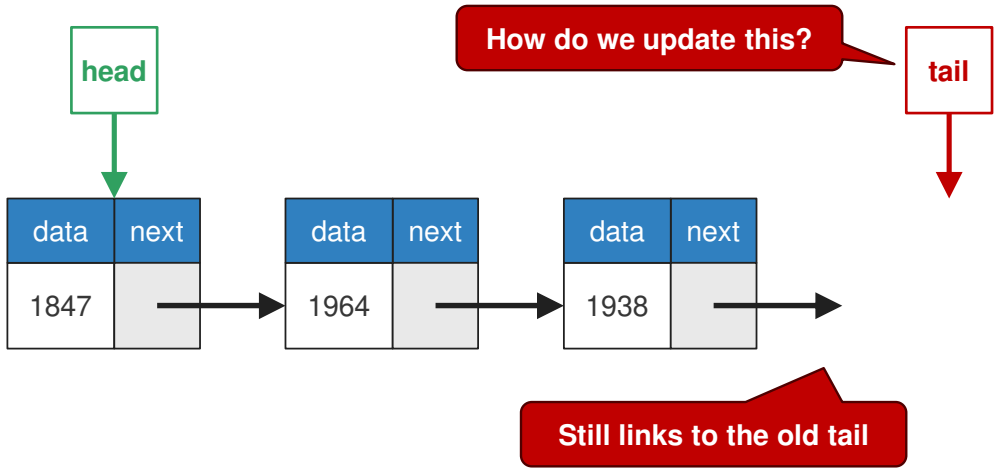
Finding Second-to-Last

- But, how do we remove the last item?
- We can find the last item immediately, but that would make second-to-last the new tail

Removing the Tail



Removing the Tail



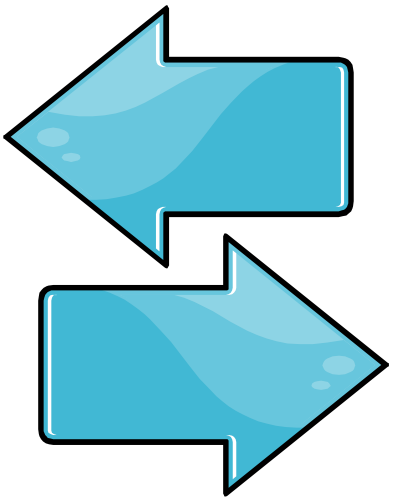
```
current = head
while (current != null)
    if (current.next == tail)
        nextToLast = current
    end if

    current = current.next        ... Go to next node
end while

... Now remove last
removed = last
nextToLast.next = null            ... Remove the old link

tail = nextToLast
```

Doubly-Linked Lists

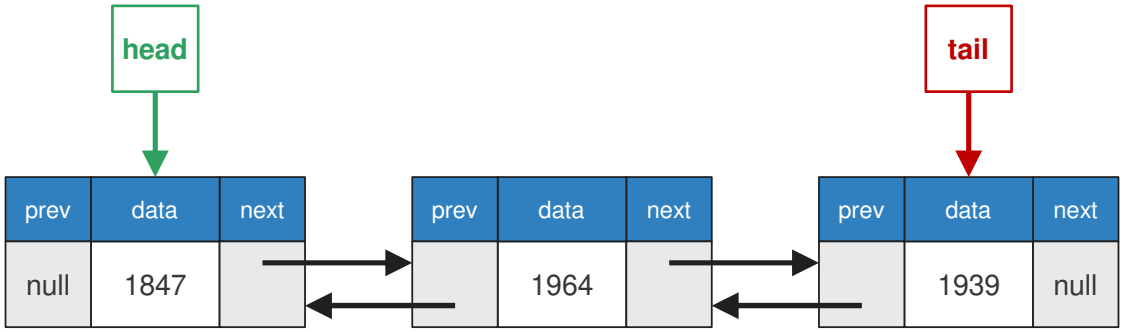
Two Way Streets

Doubly-Linked Lists

- Another variation of a linked list is the *doubly-linked list*
- As the name implies, there are two sets of links – one that points to the next node and one that points to the previous



Doubly-Linked Lists

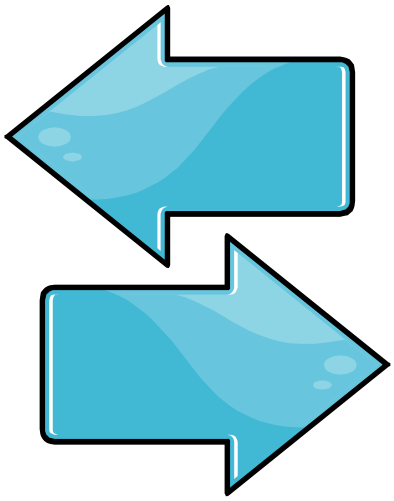


Doubly-Linked List Node

```
public class Node
{
    public Object data
    public Node prev
    public Node next
}
```

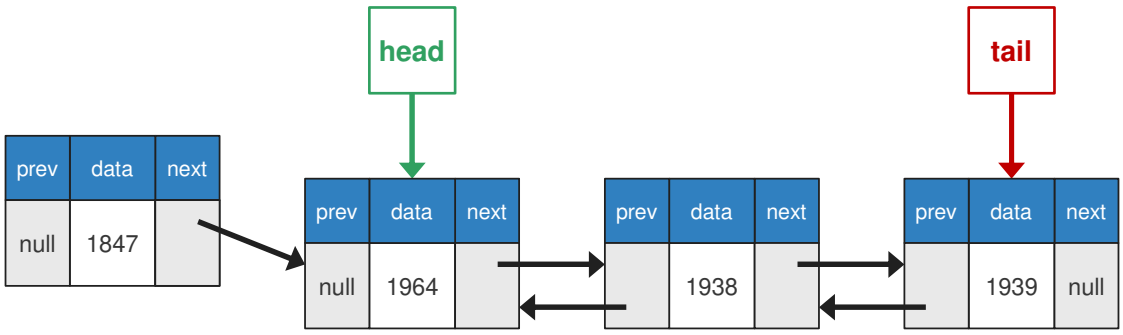
Sometimes called `last`

Doubly Linked List: Add to the Head

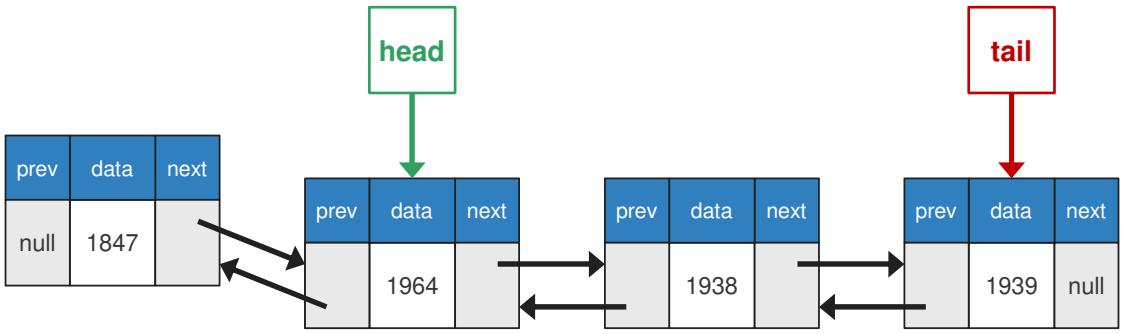


1. Link New Node to the Head
2. Link Head Back to the New Node
3. Update the Head Reference to new node

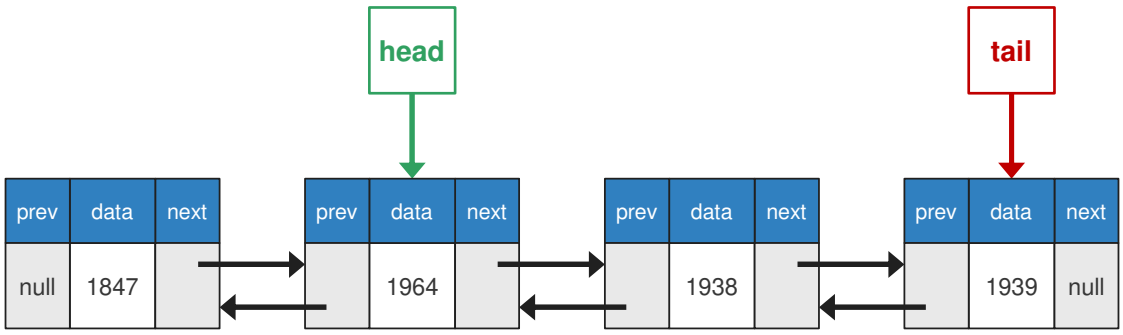
Add Head: 1. Link New Node to the Head



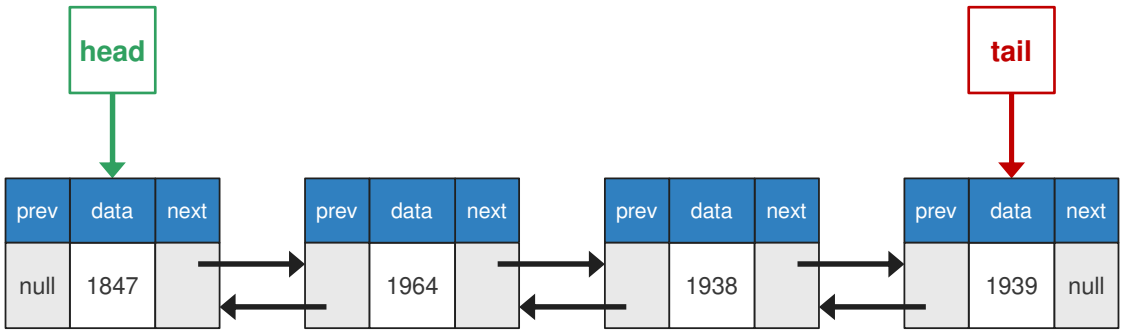
Add Head: 2. Link Head Back to the New Node



Add Head: 3. Update the Head Reference



Add Head: 3. Update the Head Reference

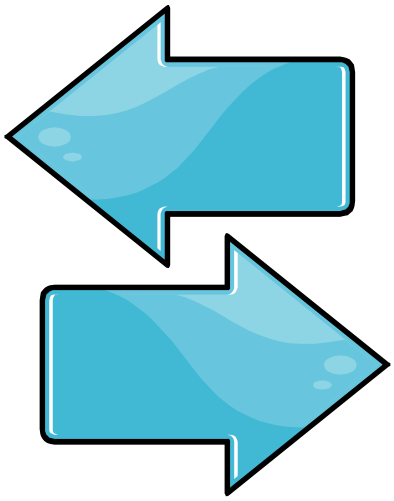


Add Head – Doubly Linked List

```
... Link the new node to old head
added.next = head

... Link the old head back to the new node
head.prev = added

... Set head to the new node
head = added
```

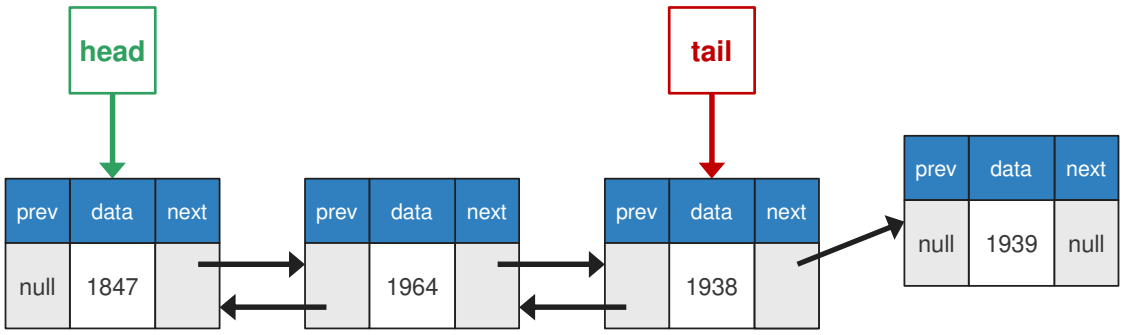
Also may be wise to check if the head == null

Doubly Linked List: Add to the Tail

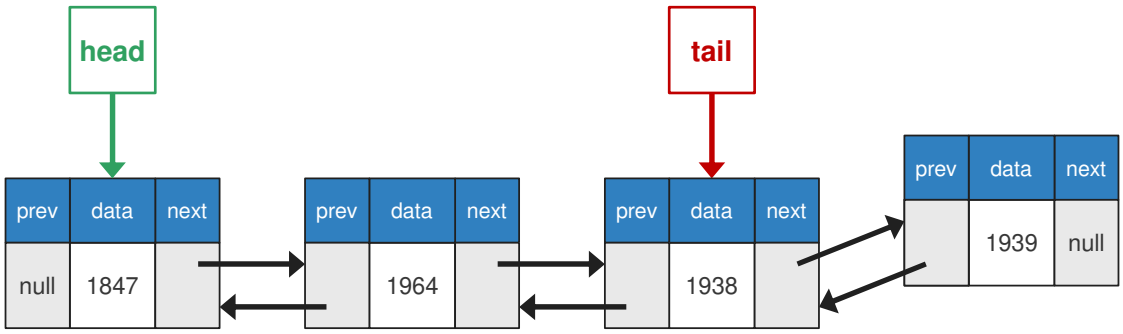


1. Link Tail to the New Node
2. Link the New Node to the Old Tail
3. Update the Tail Reference to the New Node

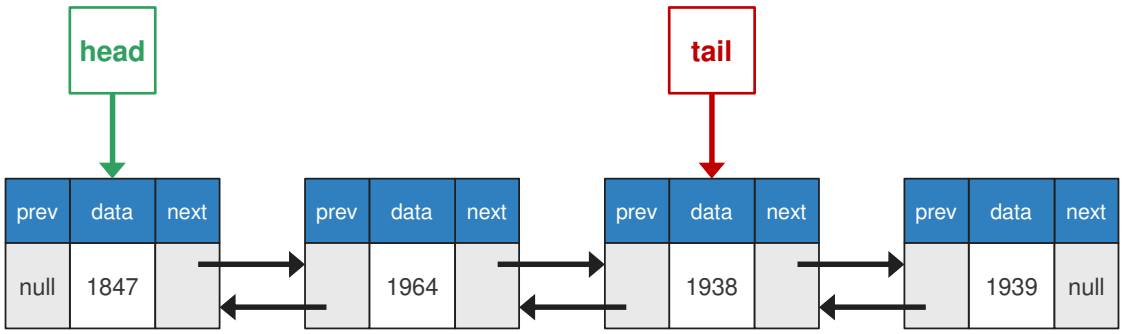
Add Tail: 1. Link Tail to New Node



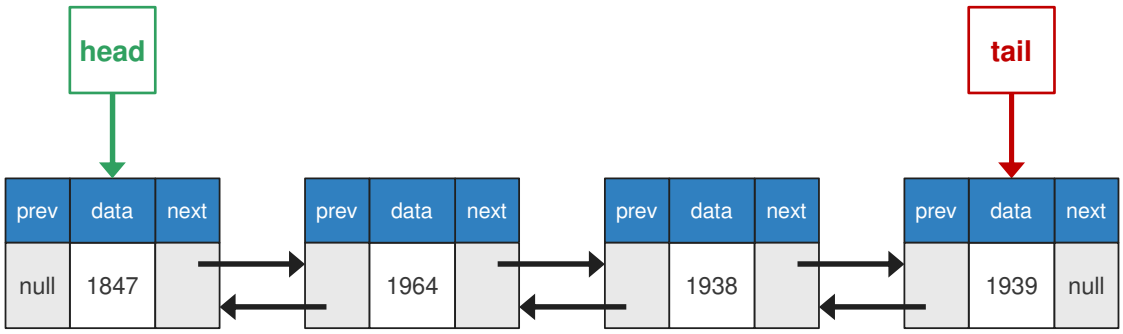
Add Tail: 2. Link the New Node to the Old Tail



Add Tail: 3. Update the Tail Reference



Add Tail: 3. Update the Tail Reference

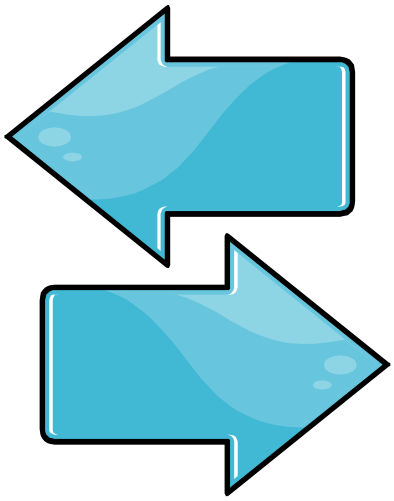


Add Tail – Doubly Linked List

```
... 1. Link the old tail to the added node
tail.next = added

... 2. Link added node back to the old tail
added.prev = tail

... 3. Set tail to the new node
tail = added
```

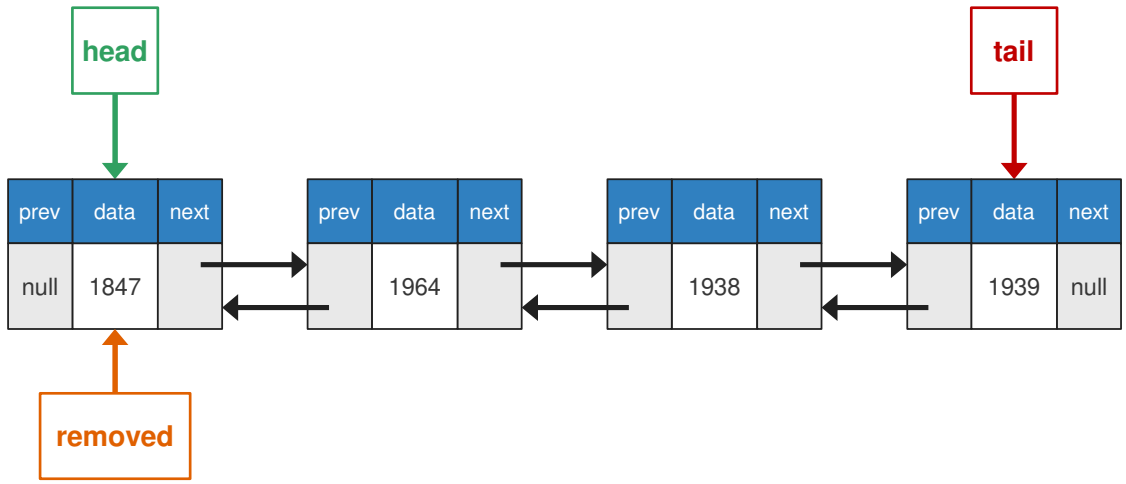
Also may be wise to check if the tail == null

Doubly Linked List: Remove Head

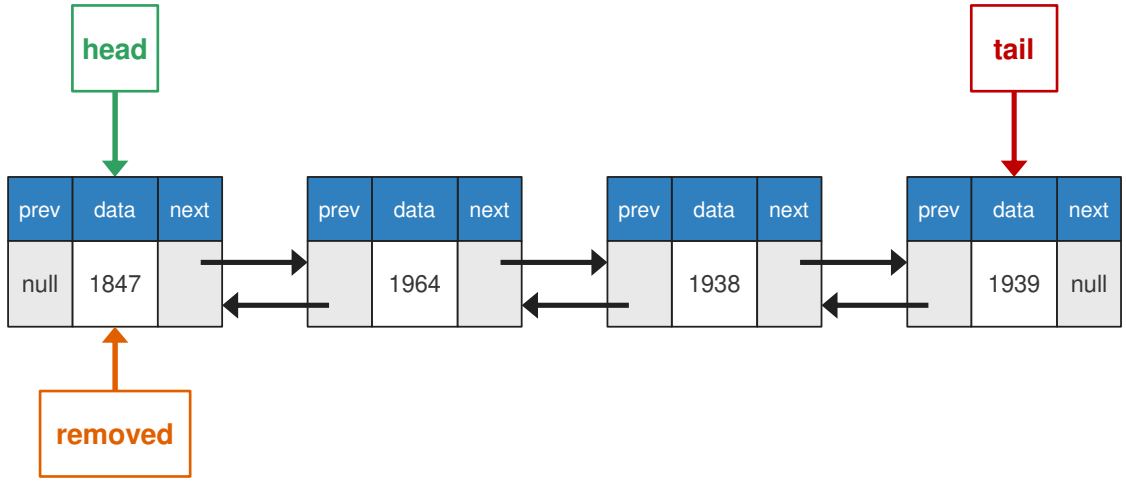


1. Save Link to the Old Head
2. Update the Head Reference to the Head's next reference
3. Remove links between Old Head and New Head

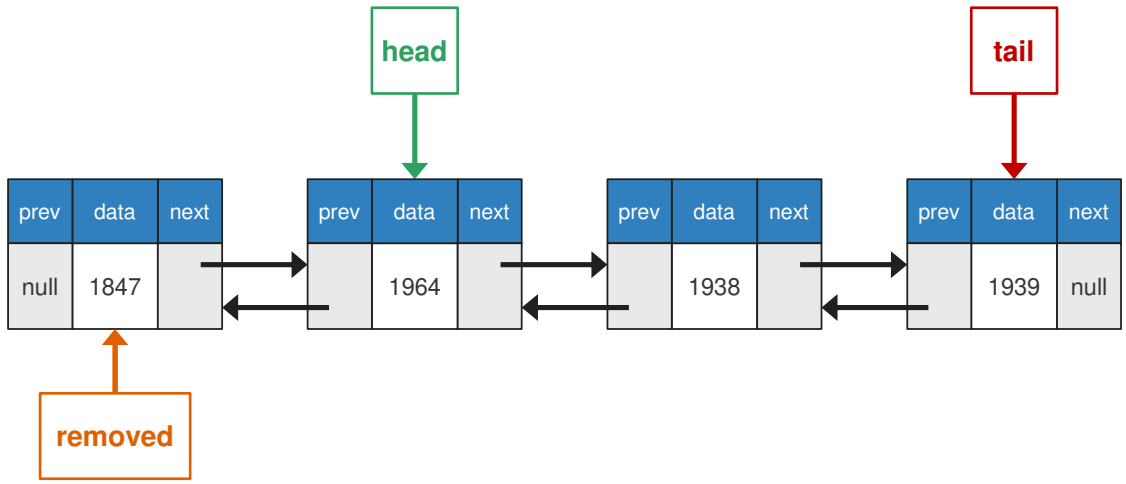
Remove Head: 1. Save Link to the Old Head



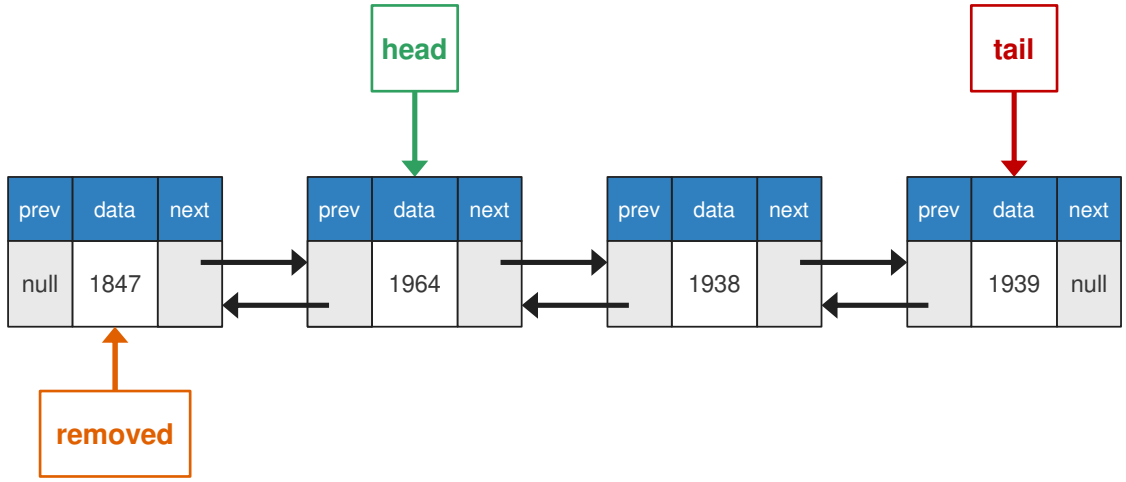
Remove Head: 2. Update the Head Reference



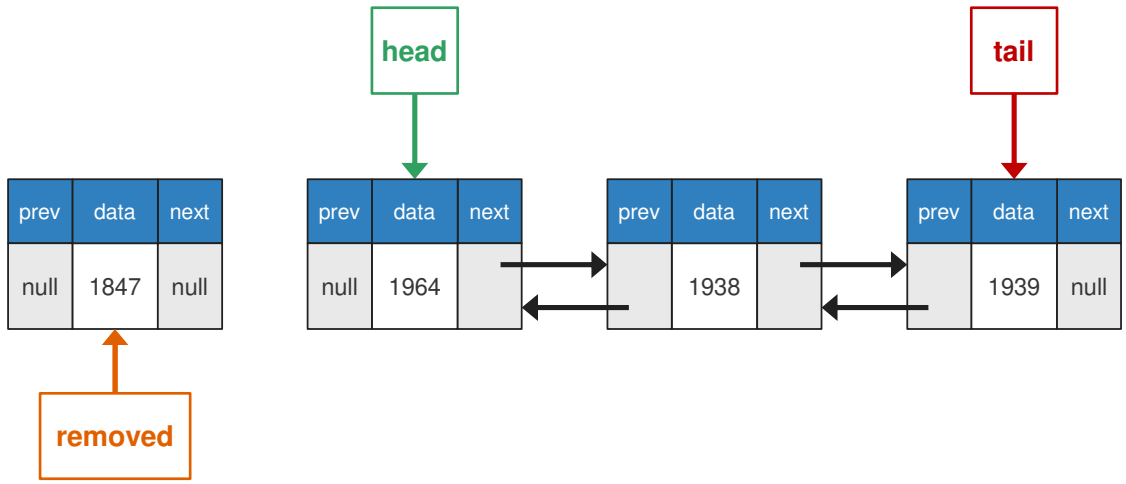
Remove Head: 2. Update the Head Reference



Remove Head: 3. Remove Links



Remove Head: Complete

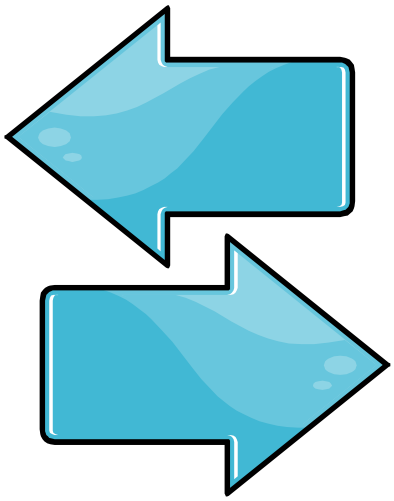


Remove Head – Doubly Linked List

```
... 1. Save a reference to the head
removed = head

... 2. Set head to the head's next link
head = head.next

... 3. Remove links between old head and new head
head.prev = null
removed.next = null
```

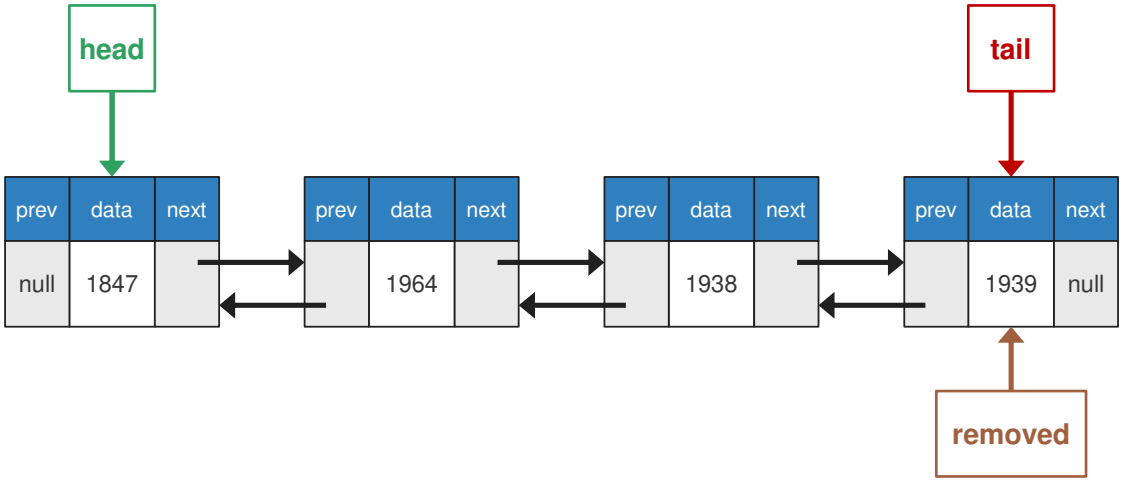
Exactly the same as a singly linked list

Doubly Linked List: Remove Tail

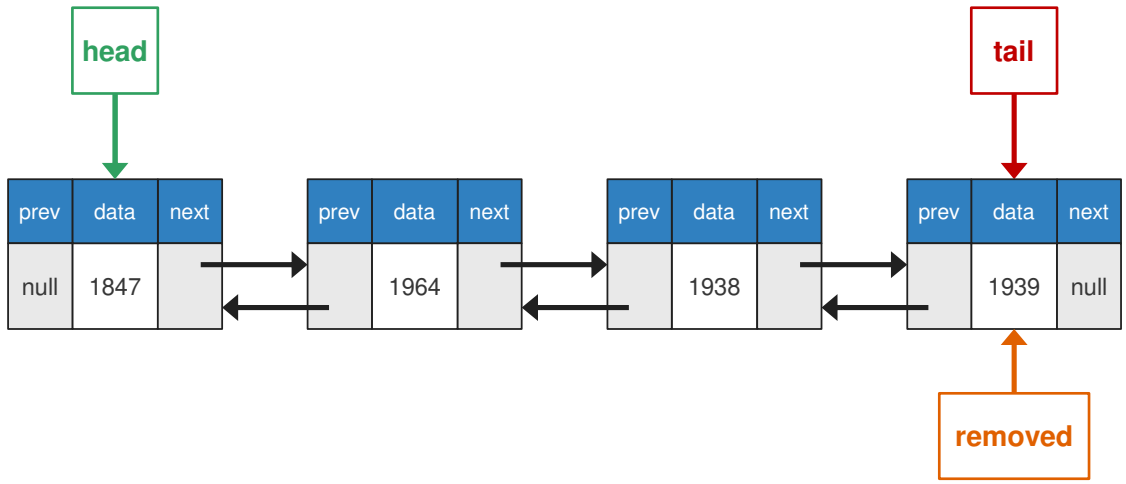


1. Save Link to the Old Tail
2. Update the Tail Reference to the previous reference of the current Tail
3. Remove links between the New Tail and the Old Tail

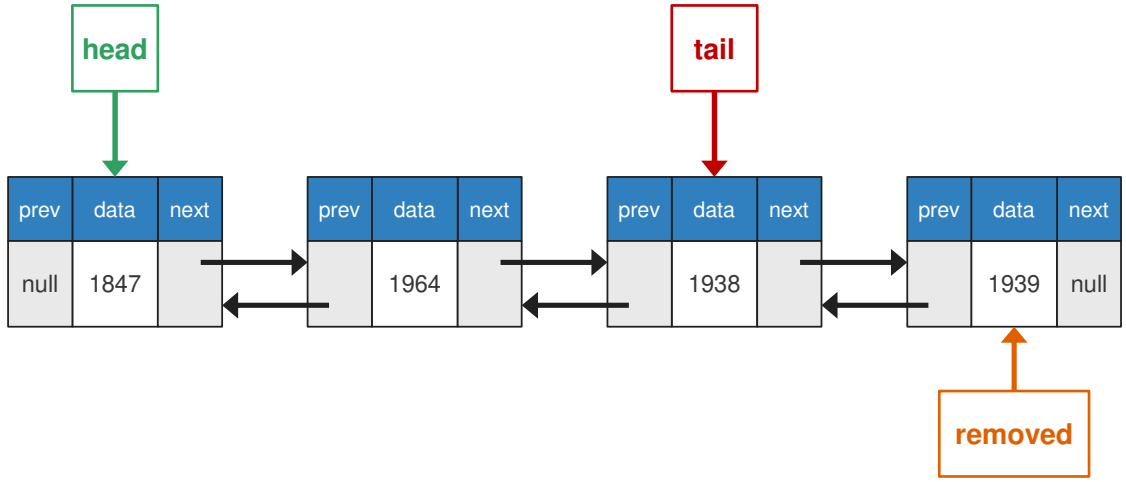
Remove Tail: 1. Save Link to the Old Tail



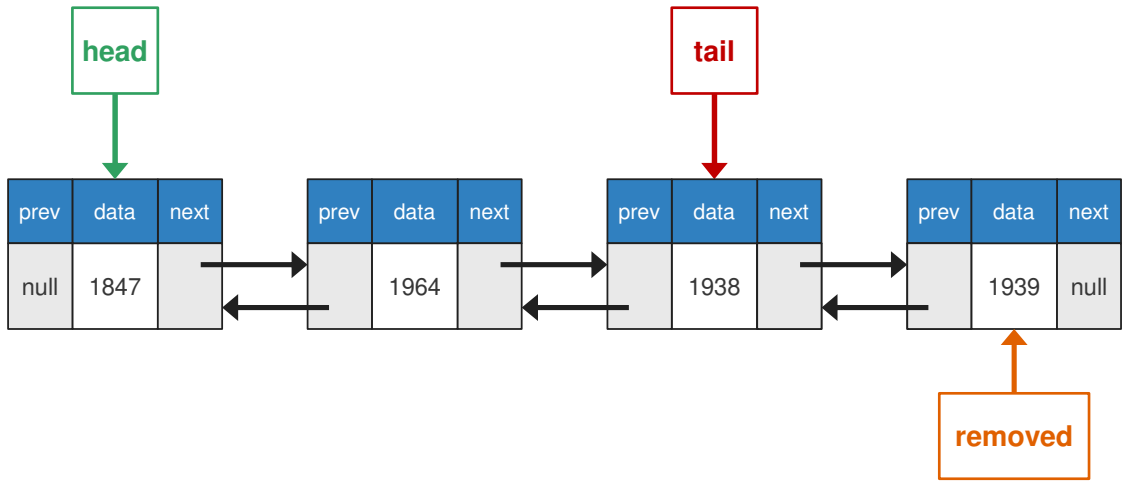
Remove Tail: 2. Update the Tail Reference



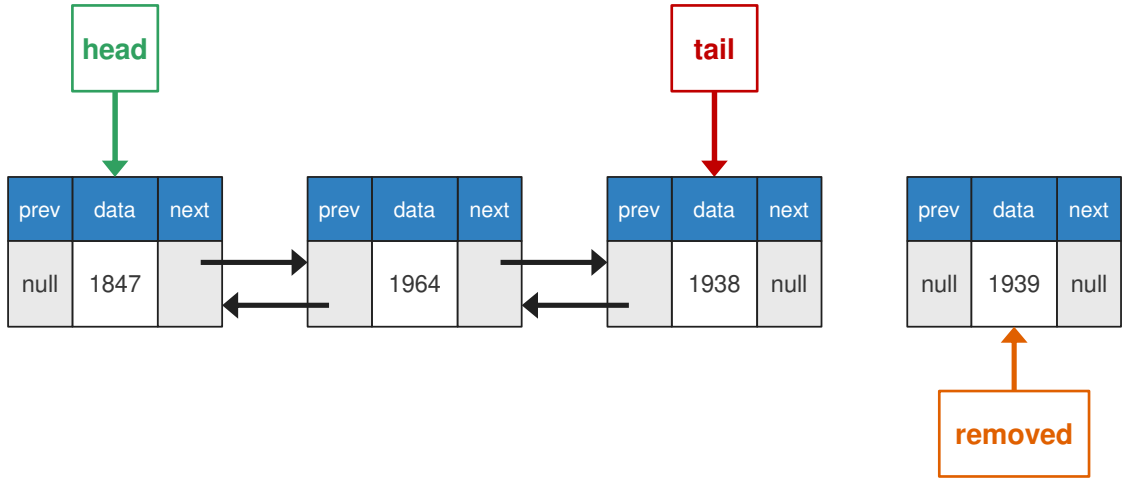
Remove Tail: 2. Update the Tail Reference



Remove Tail: 3. Remove Links

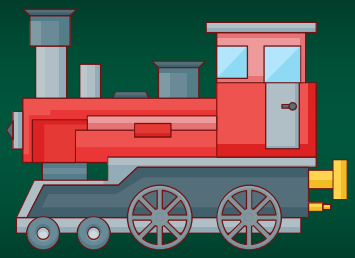


Remove Old Tail: Complete



Remove Tail – Doubly Linked List

```
... 1. Save a reference to the tail
removed = tail

... 2. Set tail to the previous of the old tail
tail = tail.prev

... 3. Remove links between old tail and new tail
tail.next = null
removed.prev = null
```

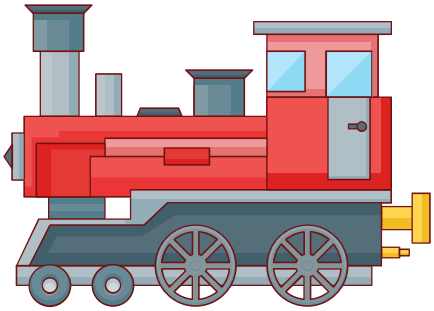



Singly-Linked List Class

Creating a train of nodes

Linked List Class

- Maintaining both a head and tail node can be a tad difficult
- So, we can place them into a LinkedList class
- Then we can write methods to add to the end (the tail) and the front (the head)



Linked List Class

```
class LinkedList
    ... ← Node class defined here

    private Node head
    private Node tail

    ... ← Methods here
end class
```

Linked List Class

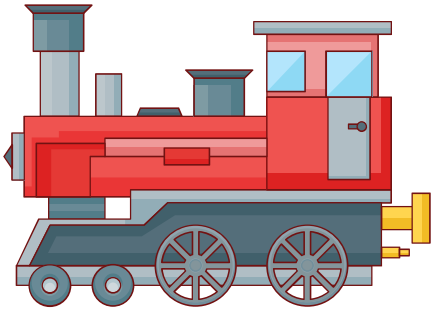
```
public function AddHead(Object data)
    added = new Node(data)

    if (this.head == null)          ... List is empty. Add first node.
        this.head = added           ... Link both
        this.tail = added
    else
        added.next = this.head      ... Link new node to the current head
        this.head = added           ... Now the new node is the head
    end if
end function
```


Linked List Class

```
public function AddTail(Object data)
    added = new Node(data)

    if (this.head == null)          ... List is empty. Add first node.
        this.head = added           ... Link both
        this.tail = added
    else
        this.tail.next = added      ... Link new node to the current head
        this.tail = added           ... Now the new node is the head
    end if
end function
```

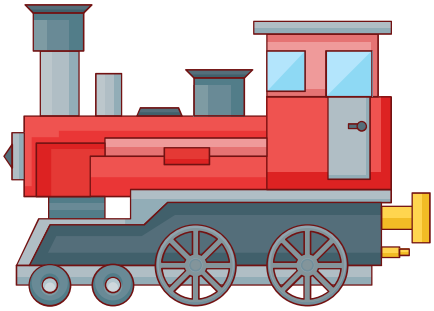
Linked List Class

- Now that we have compensated for the head/tail being null, we can also a method to remove the head
- But there are more cases that need to be considered



Three Difference Cases

1. There are no nodes
2. There is only one node (the head and tail are the same)
3. There are 2+ nodes (the head and tail are different)



```
public function RemoveHead() return Object
    Object result

    if (this.head == null)                ... Case 1: List is empty
        result = null                     ... We could also throw an error
    else if (head == tail)                ... Case 2: Just one node.
        result = this.head.data           ... or tail - either or works.
        head = null                       ... Deference both
        tail = null
    else                                  ... Case 3: Two or more nodes.
        result = this.head.data
        this.head = this.head.next        ... move head to the second node
    end if

    return result
end function
```

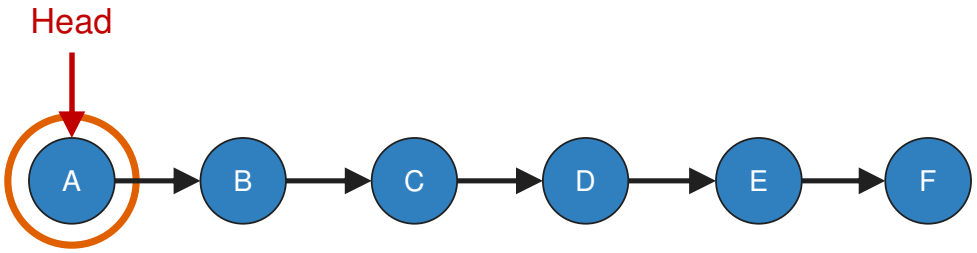
Linked List Big-O

How Good Is This?

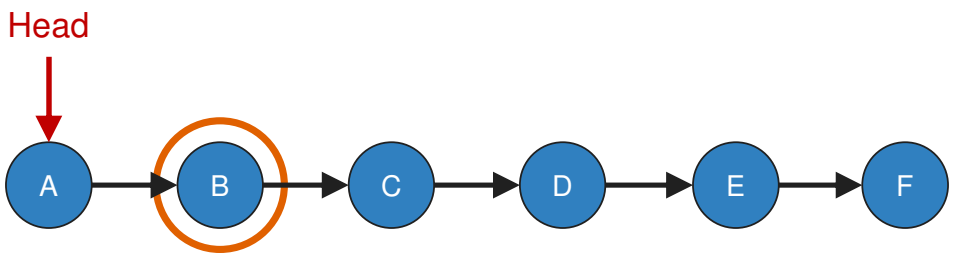
Linked List Data Structure

- Unlike arrays, where the element can be found using a calculation, linked-lists require the list to be traversed
- So, finding an item in a linked list requires **O(n)**

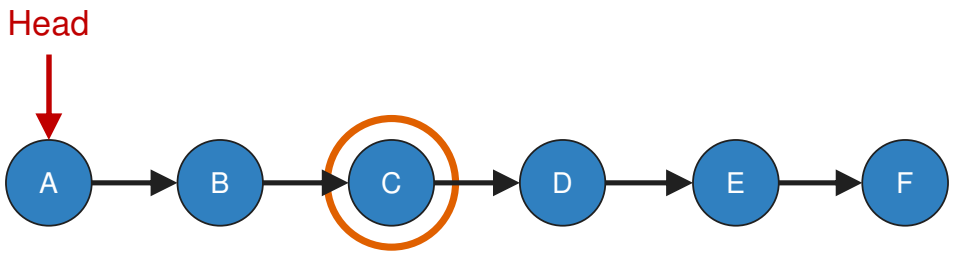
Single-Linked List – Find D



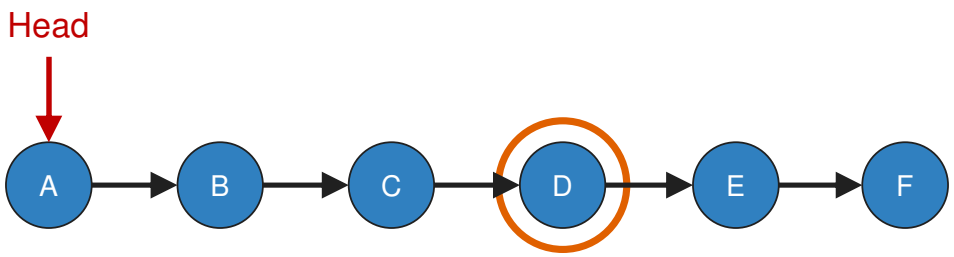
Single-Linked List – Find D



Single-Linked List – Find D



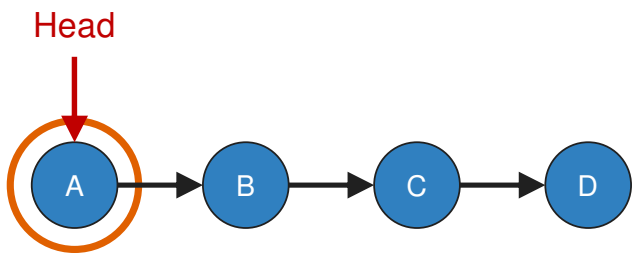
Single-Linked List – Find D



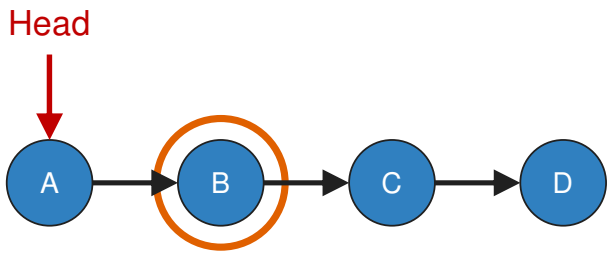
Head and Tail Nodes

- Linked lists maintain a link to the head node
- Often, in well-written linked lists, a link to the tail node is also maintained
- Why? It has a huge impact on time complexity

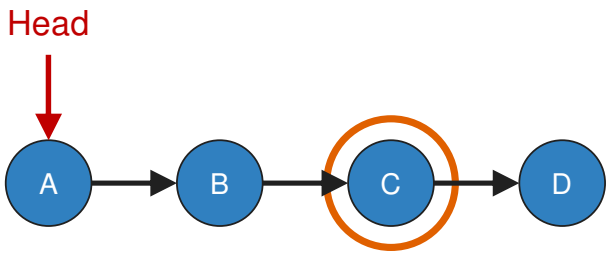
Append Value – No Tail Node



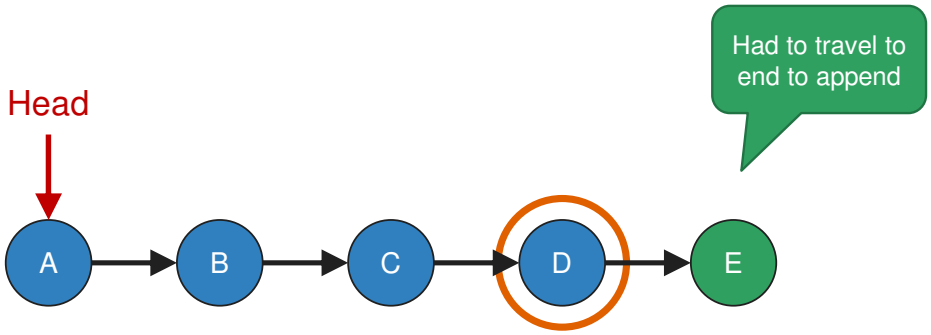
Append Value – No Tail Node



Append Value – No Tail Node



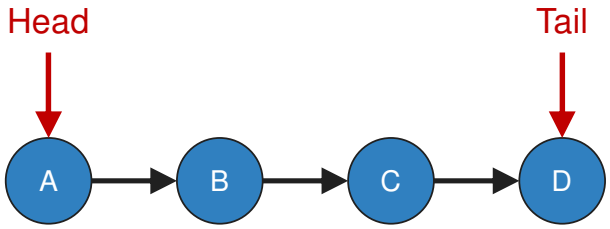
Append Value – No Tail Node



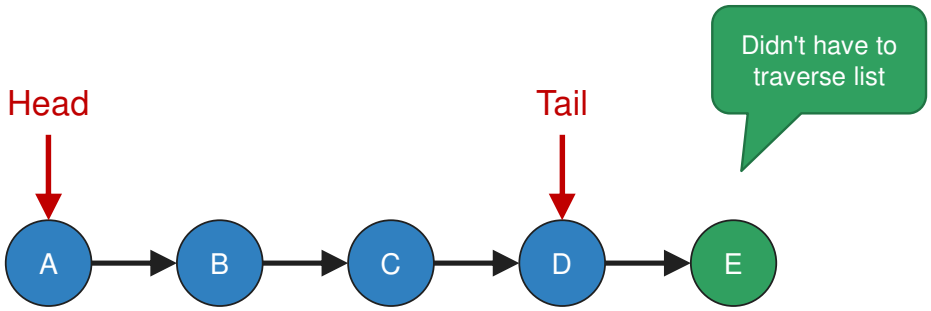
Head and Tail Nodes

- Without a tail node, the entire list must be traversed to find the end
- This will require **O(n)**
- Adding a tail node, will decrease it to **O(1)**

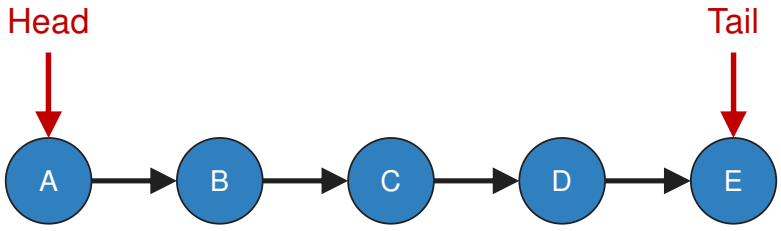
Append Value – With Tail Node



Append Value – With Tail Node



Append Value – With Tail Node



Use a Tail Node!

- Unless you are only appending nodes at the head of a linked list, maintain a tail node
- For **all** the examples used in these slides… assume the linked list has a tail node

Auxiliary Storage in Linked Lists

- Unlike arrays, linked lists must store the "next" links between nodes
- So, the *auxiliary storage* overhead is O(n)
  - …which is usually the size of an address
  - 64-bit system → 8 bytes

Big-O: Test Your Might…

```
LinkedList list

for(i = 0 i < list.Count i++)
{
   total += list.FindAtPosition(i)
}
```

O(n)

O(n)

$O(n^2)$

Iterators

- To avoid accidental $O(n^2)$, major programming languages support *iterator objects*
- They store information about the current state (e.g. a node) when data is being are <u>sequentially</u> read

Iterators

- Iterators maintain O(n) for sequentially accessing all the list's elements
- This is the purpose of the For-Each Statement
- Notation varies greatly between languages (when they are supported)

Dynamic Allocated Array vs. Linked List

Operation	Dynamic Array	Singly Linked List	Doubly Linked List
Find (random)	O(1)	O(n)	O(n)
Insert (random)	O(n)	O(n)	O(n)
Add first/last	O(n)	O(1)	O(1)
Remove first	O(n)	O(1)	O(1)
Remove last	O(n)	O(n)	O(1)
Auxiliary storage	O(1)	O(n)	O(n)